

TP N°1: arVault

[75.73/TB034] Arquitectura de Software
CURSO: 01-Calónico
Primer cuatrimestre de 2026

Alumno/a:	FARALL CASADO, Tomás
Número de padrón:	102628
Email:	...

Alumno/a:	VACCARELLI, Santiago
Número de padrón:	106051
Email:	svaccarelli@fi.uba.ar

Alumno/a:	JANG, LucasLucas Jang
Número de padrón:	109151
Email:	...

Alumno/a:	KRESTA, Facundo Ariel
Número de padrón:	110857
Email:	facundoarielkresta@gmail.com

Índice

1	Introducción	2
1.1	Contexto	2
1.2	Objetivo	2
2	Atributos Importantes	3
2.1	Atributos de Calidad Clave	3
2.2	Crítica a las Decisiones de Diseño Originales	3
2.3	Desempeño y Pruebas de Carga del Caso Base	3
2.4	Mejoras implementadas (Layered Architecture)	4
2.5	Modificaciones Propuestas (Tácticas y Trade-offs)	4
2.5.1	Impacto y Trade-offs a constatar	4
3	Decisiones de Diseño (C&C)	5
3.1	Evolución: Del Monolito Acoplado a Arquitectura en Capas	5
3.2	Variables en memoria	8
3.3	Persistencia lazy	8
3.4	Persistencia en archivos json	8
3.4.1	Función transfer() mockeada de forma secuencial	8
3.5	Instancia única sin redundancia ni healthcheck	8
3.6	Falta de logs de operaciones	8
4	Métricas: Caso Base (Lectura de Rates)	9
5	Conclusión (Parcial)	11

1. Introducción

1.1. Contexto

La startup **arVault** es una fintech que opera una billetera digital, de reciente creación. Su fundador, un desarrollador aficionado y entusiasta con algo de dinero y muchas ideas, consciente de que la clave de su negocio es la implementación de su **backend**, tercerizó el desarrollo de las aplicaciones para dispositivos móviles, y se concentró en implementar rápidamente (bajo la consigna *fake it till you make it*¹) un núcleo de servicios que le permitieran conseguir más fondos a través de inversores.

Luego de la ronda inicial, los primeros fondos fueron utilizados, no para robustecer la arquitectura existente, sino para agregar más funcionalidad. Una de estas funcionalidades nuevas permite abrir cuentas en distintas monedas (que se respaldan en bancos existentes), y realizar operaciones de cambio entre éstas. El diferencial de **arVault** es tener la tasa de cambio más conveniente dentro de las aplicaciones que proveen este servicio. Para ganar usuarios, las tasas de cambio no tienen *gap* entre el valor de compra y de venta.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas. Estos reclamos llegaron en el peor momento, dado que **arVault** necesita captar más inversiones y, debido a la caída de la reputación y las reviews negativas, los potenciales inversores se niegan a aportar más fondos si no se realiza una auditoría y se mejora el servicio.

Frente a este reclamo, **arVault** decide contratar a un grupo de arquitectos para que evalúen, propongan e implementen soluciones que mejoren los atributos de calidad del servicio de cambio de monedas.

1.2. Objetivo

El objetivo de este informe es presentar una evaluación de la arquitectura actual del servicio de cambio de monedas, identificar los atributos de calidad que se ven afectados (ver Apartado 2) por los reclamos de los usuarios, y proponer decisiones de diseño que puedan mejorar dichos atributos (Apartado 3). Además, se presentarán métricas en el Apartado 4 para evaluar el impacto de las decisiones propuestas y se discutirán las conclusiones obtenidas a partir del análisis realizado. Finalmente, se incluirá una sección de conclusiones (Apartado 5) que sintetice los hallazgos y recomendaciones para **arVault**.

¹Expresión popular que refiere a simular confianza o habilidades hasta realmente adquirirlas.

2. Atributos Importantes

2.1. Atributos de Calidad Clave

Considerando la naturaleza financiera y transaccional del servicio provisto (*arVault*, un *exchange* de criptomonedas y dinero fiat), se han determinado como centrales los siguientes Atributos de Calidad (QAs):

- **Confiabilidad (Reliability) e Integridad de Datos:** En un flujo transaccional donde operan saldos económicos (balances, transferencias), el sistema no debe perder información bajo ninguna circunstancia (por ejemplo, ante la caída del proceso de Node.js). Todas las transacciones confirmadas deben reflejarse en un medio persistente seguro.
- **Desempeño (Performance):** El servicio debe procesar múltiples operaciones concurrentes de *exchange* con un tiempo de respuesta (latencia) adecuado y un nivel alto de *throughput* bajo momentos de mayor demanda.
- **Escalabilidad (Scalability):** Previendo un escenario con cantidad creciente de usuarios operando simultáneamente, el servicio debe tener la capacidad de ser replicado (escalamiento horizontal) u optimizado para digerir la carga sin generar cuellos de botella irreparables.

2.2. Crítica a las Decisiones de Diseño Originales

La arquitectura y código del “caso base” presentan decisiones de diseño cuestionables desde el punto de vista arquitectónico, las cuales impactan drásticamente en los QAs:

- **Manejo del Estado y Confiabilidad (Reliability):** El módulo `state.js` carga inicialmente todo el estado a memoria desde archivos locales JSON y emplea un ciclo `setInterval` cada 1 a 5 segundos (`scheduleSave`) para reescribir el archivo entero de forma asincrónica. Si el contenedor o el proceso muere de manera abrupta, las operaciones (*exchanges*) realizadas desde la última escritura hasta el instante de la falla se pierden irremediablemente, rompiendo la confiabilidad del servicio y dejando perfiles inconsistentes.
- **Condiciones de Carrera (*Race conditions*):** En el código de `exchange.js`, el diseño de la asincronía está acoplado con chequeos inválidos. Por ejemplo, al evaluar `counterAccount.balance >= counterAmount`, se bloquea el hilo utilizando `await transfer(...)`. Si entran múltiples peticiones concurrentes antes de que finalicen las transferencias virtuales y se efectúen los respectivos `baseAccount.balance += ...` y `-=`, el sistema permitirá ejecutar *exchanges* con fondos insuficientes, violando reglas de dominio críticas de consistencia y seguridad.
- **Escalabilidad:** El estado dependiente de la memoria de la aplicación en nodo y la escritura de archivos simples dentro del propio contenedor, rompe uno de los pilares de arquitecturas en la nube (stateless). Al no poseer un mecanismo de sincronía centralizado (como una base de datos ACID), es imposible desplegar y sincronizar múltiples instancias de la aplicación (escalamiento horizontal) tras un balanceador de carga. Cada instancia trabajará con un estado divergente de balances.
- **Testabilidad (Testability) y Modificabilidad:** El alto acoplamiento y el almacenamiento manual limitan fuertemente la escritura de pruebas aisladas, limitándose el sistema a un escenario fraccionario.

2.3. Desempeño y Pruebas de Carga del Caso Base

Para evidenciar el comportamiento de los atributos de **Performance** sobre la arquitectura base, se vuelve indispensable contar con métricas precisas. Tal como se puede apreciar en la Sección 4, al ejecutar una prueba de carga sobre el endpoint de consultas (`GET /rates`), el sistema denota una latencia mediana excelente (~ 3 ms, *p99* de 13,1 ms) y una tasa de respuesta estable, ya que es una mera consulta a la memoria (operación no bloqueante y sin I/O pesado).

Aun así, este comportamiento favorable es dependiente de que las operaciones no transaccionen (como `/exchange`).

2.4. Mejoras implementadas (Layered Architecture)

Se ha introducido un refactor completo del servicio pasando de un **Monolito Acoplado** a una **Arquitectura en Capas (Layered Architecture)**. Esta alteración mejora contundentemente los atributos de **Modificabilidad** y **Testabilidad**, separando las responsabilidades de enrutamiento (Controllers), lógica de dominio (Services) y persistencia (Repositories).

No obstante, esta reestructuración lógica **no soluciona los problemas de fondo en Escalabilidad y Confiabilidad**. Si bien ahora la persistencia está abstraída en el 'stateManager', los datos siguen viviendo en memoria e impactándose asincrónicamente mediante 'fs' en JSON locales.

2.5. Modificaciones Propuestas (Tácticas y Trade-offs)

Para solucionar verdaderamente los desperfectos de los QA clave (Confiabilidad y Escalabilidad), proponemos dos líneas de modificación basadas en tácticas arquitectónicas concretas que pueden insertarse fácilmente gracias a la nueva capa de Repositorios:

1. **Externalización del Estado (Data Abstraction y Transaction Management):** Modificar el mecanismo interno de los **Repositories** para consumir un servicio externo en tiempo real (Base de Datos Redis/Mongo o Relacional). Esto aislará el estado y volverá a los contenedores Node.js *Stateless*.
2. **Escalamiento Horizontal y Balanceo (Concurrency y Multiple Instances):** Una vez resuelto el estado externalizado, en la infraestructura se propondrá habilitar nodos en *Nginx* como Reverse Proxy de modo que el trabajo se distribuya sobre múltiples réplicas (*Round-Robin*).

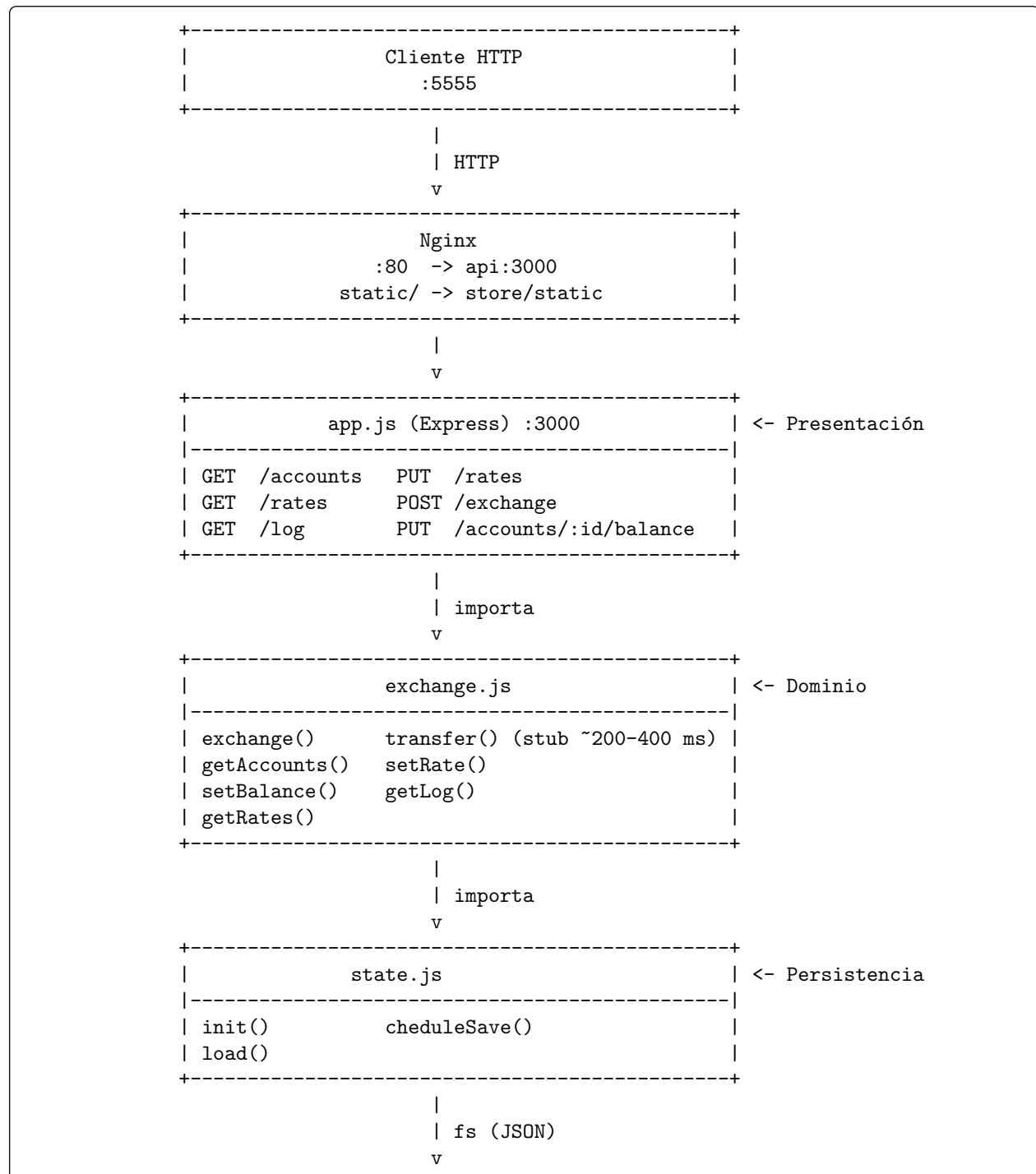
2.5.1. Impacto y Trade-offs a constatar

[NOTA: Aquí se colocaría de qué manera el cambio de DB alteró la latencia sobre el endpoint /exchange o /rates. Con pruebas sobre el endpoint /exchange, con el escenario .^{exchange}.yaml, incluirlas frente a las nuevas de DB y analizar el trade-off "Latencia vs Consistencia/Escalabilidad".]

3. Decisiones de Diseño (C&C)

3.1. Evolución: Del Monolito Acoplado a Arquitectura en Capas

Originalmente, el servicio constaba de un monolito de una sola capa. Había tres archivos con roles distintos (`app.js` operando de HTTP handler, `exchange.js` manejando lógica de negocio, y `state.js` interactuando con la persistencia en disco) que se encontraban acoplados directamente sin una clara inyección de dependencias ni definición de interfaces seguras. Todo corría en el hilo de un único proceso de Node.js, penalizando *Testability*, *Maintainability* y *Modifiability*. El caso base se ilustra debajo:



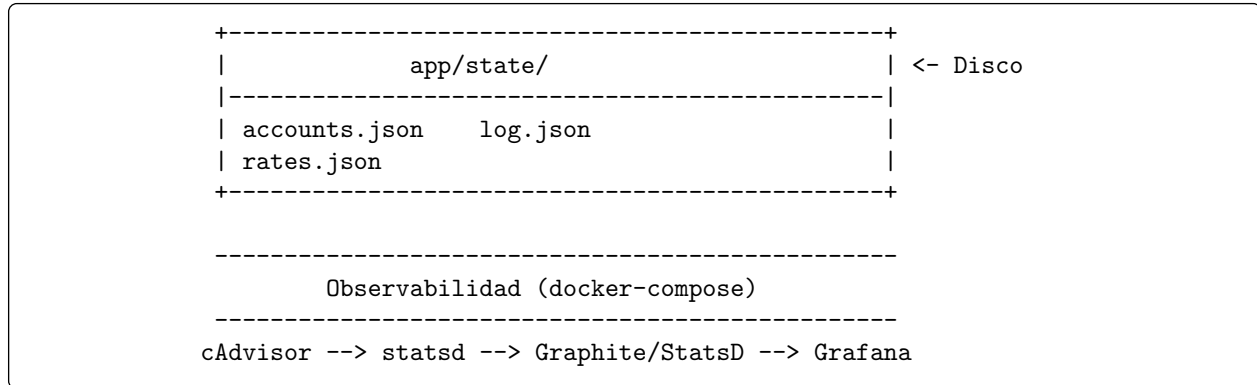
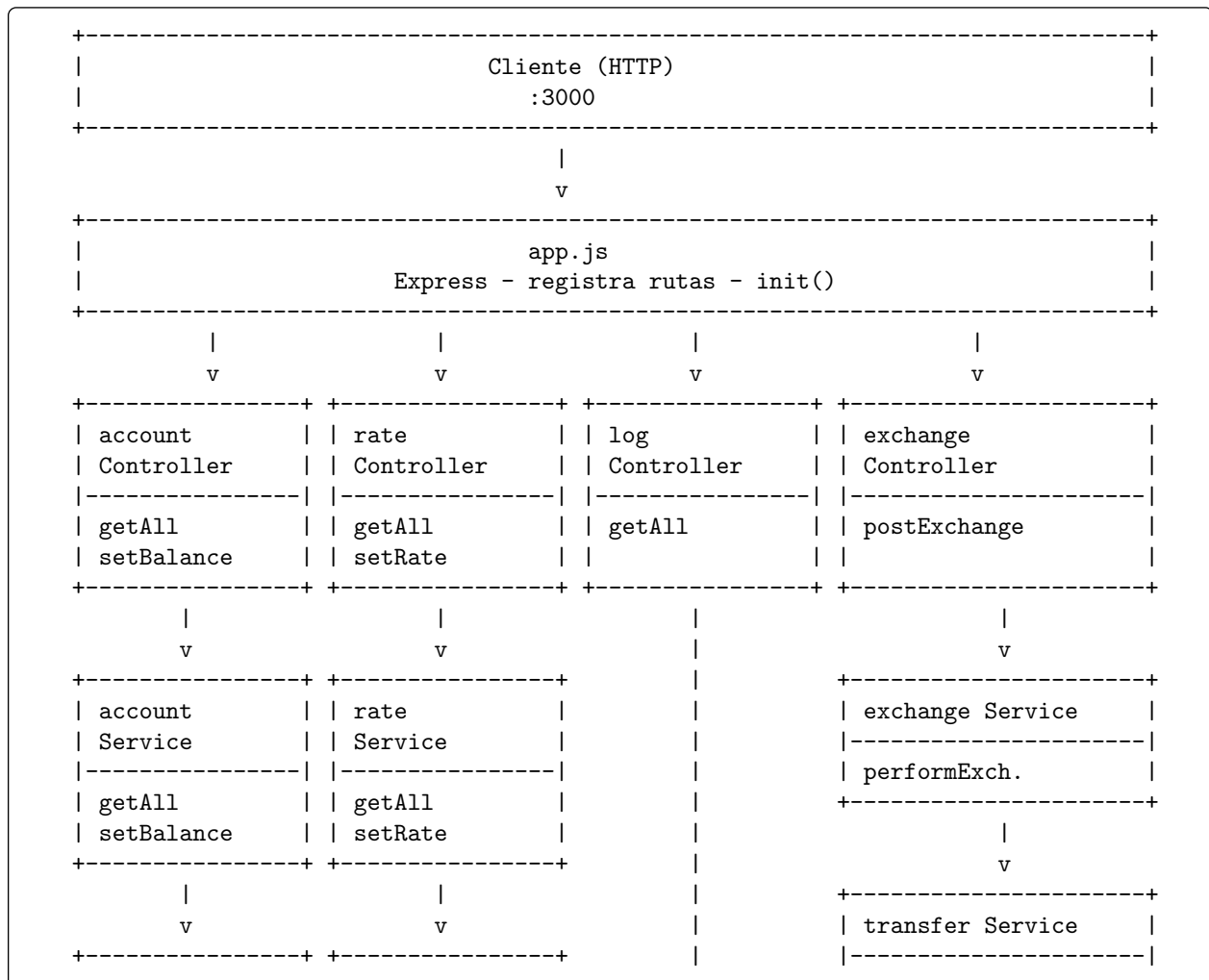


Diagrama 3.1: Arquitectura monolítica del sistema

En `exchange.js`: `accounts = stateAccounts();` // alias al mismo array en memoria `rates = stateRates();` `log = stateLog();` `state.js` guarda esas mismas referencias y las persiste con `scheduleSave`. `exchange.js` muta `counterAccount.balance`, `baseAccount.balance`, `log.push(...)` directamente — sin pasar por ningún método de `state.js`. Es decir, la capa de dominio escribe en la capa de persistencia *bypaseando* su interfaz.

`app.js` también mezcla responsabilidades `app.js`: hace validación de input, llama al servicio Y decide el formato de respuesta todo en el mismo handler. No hay capa de controlador separada.

Después:



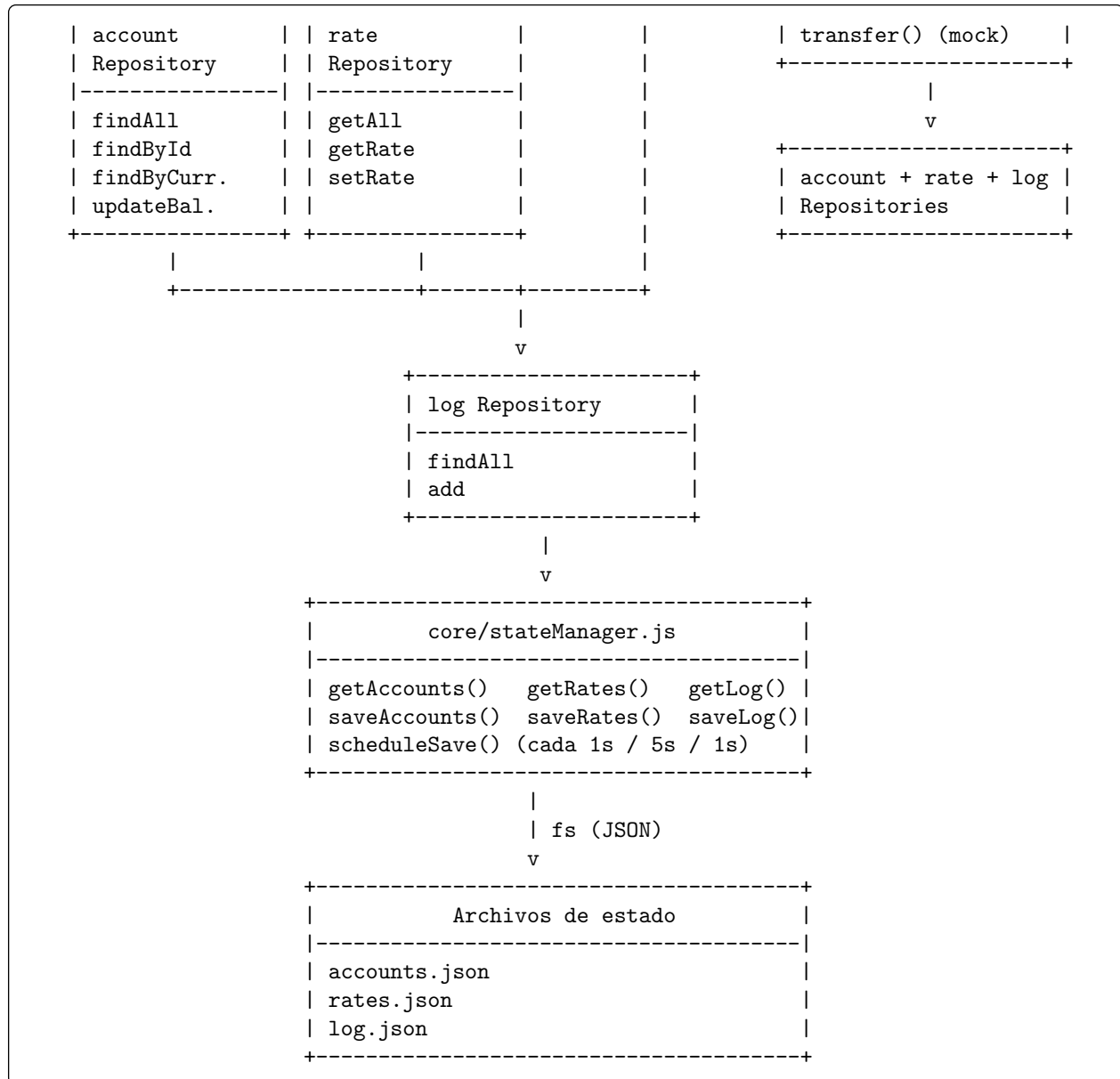


Diagrama 3.2: Arquitectura monolítica luego de la mejora

Capa	Archivos	Responsabilidad
Presentación	controllers/ + app.js	HTTP, validación de estructura, mapeo de errores a status codes
Dominio	services/	Reglas de negocio, validaciones de dominio (balance <= 0, etc.)
Persistencia	repositories/ + core/stateManager.js	Acceso y mutación del estado, lectura/escritura a disco
Utils	utils/errors.js + utils/validators.js	Tipos de error compartidos, funciones de validación puras

Tabla 3.1: Capas de la aplicación y sus responsabilidades

3.2. Variables en memoria

Accounts, rates y log son variables JavaScript dentro del proceso. Leer es instantáneo (bueno para performance), pero hace imposible escalar horizontalmente porque cada instancia tendría su propio estado desincronizado. Además genera la race condition: dos requests concurrentes pueden leer el mismo balance, pasar ambos el check de fondos, y producir doble gasto.

3.3. Persistencia lazy

El estado se guarda a disco cada 1-5 segundos en lugar de en cada operación. Simplifica el código y no bloquea requests con I/O, pero si el proceso muere entre guardados se pierden transacciones, violación de durabilidad.

3.4. Persistencia en archivos json

Todo vive en archivos JSON locales. Sin transacciones atómicas, sin estado compartible entre instancias, y con riesgo de corrupción si el archivo falla.

3.4.1. Función transfer() mockeada de forma secuencial

```
1 async function transfer(fromAccountId, toAccountId, amount) {  
2   const min = 200;  
3   const max = 400;  
4   return new Promise((resolve) =>  
5     setTimeout(() => resolve(true), Math.random() * (max - min + 1) + min));  
6 }
```

Código 1: Ejemplo de función transfer()

La función ignora todos sus parámetros y siempre resuelve true tras un delay aleatorio de 200–400 ms.

3.5. Instancia única sin redundancia ni healthcheck

Impacto sobre Availability La API es un Single Point of Failure (SPOF). Un crash del proceso Node.js (error no capturado, OOM, señal del OS), un reinicio por deploy o cualquier excepción no manejada detiene completamente el servicio. Sin 'healthcheck' declarado, Docker y Nginx no pueden detectar automáticamente que el contenedor dejó de responder. Nginx continuará enrutando requests al upstream caído, retornando 502 Bad Gateway a todos los usuarios.

Impacto sobre User-Perceived Performance Sin distribución de carga entre instancias, todos los requests concurrentes comparten el único event loop de Node.js. Las esperas de 'await transfer()' (400–800 ms por request) bloquean efectivamente el procesamiento del resto de requests que llegan mientras tanto.

Impacto sobre Scalability Nginx no está configurado para balancear hacia múltiples backends. Aunque se pudieran levantar réplicas ('docker compose up --scale api=3'), el upstream hardcodeado a 'exchange-api-1' ignorará las instancias adicionales. Para habilitar el balanceo se requiere modificar la configuración de Nginx.

3.6. Falta de logs de operaciones

Impacta en el no repudio (security) y la falta de poder hacer una auditoría.

4. Métricas: Caso Base (Lectura de Rates)

En esta sección se presentan los resultados correspondientes al test de carga base ejecutando el script `perf/rates.yaml`. Las métricas documentadas y la imagen extraída del dashboard de Grafana (`perf/dashboard.json`) retratan el comportamiento del sistema *previo* a la aplicación de cambios relacionados con tácticas de Escalabilidad, pero nos exponen los tiempos crudos de lectura.

```

http.codes.200: ..... 390
http.downloaded_bytes: ..... 42510
http.request_rate: ..... 3/sec
http.requests: ..... 390
http.response_time:
  min: ..... 2
  max: ..... 30
  mean: ..... 3.6
  median: ..... 3
  p95: ..... 5
  p99: ..... 13.1
http.response_time.2xx:
  min: ..... 2
  max: ..... 30
  mean: ..... 3.6
  median: ..... 3
  p95: ..... 5
  p99: ..... 13.1
http.responses: ..... 390
vusers.completed: ..... 390
vusers.created: ..... 390
vusers.created_by_name.Rates (/rates): ..... 390
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 4
  max: ..... 74
  mean: ..... 6.5
  median: ..... 5.3
  p95: ..... 10.5
  p99: ..... 27.9

```

Diagrama 4.1: Métricas obtenidas de la prueba de rendimiento



Imagen 4.1: Métricas obtenidas de la prueba de rendimiento

5. Conclusión (Parcial)

A lo largo del análisis de la arquitectura inicial (*Caso Base*) del *exchange*, se ha constatado que el sistema evidenciaba profundas deficiencias estructurales en sus Atributos de Calidad más críticos, notablemente en relación al eje de **Confiabilidad, Escalamiento e Integridad Transaccional**.

Los pasos ejecutados hasta el momento (el refactor a una *Layered Architecture*) han mitigado parcialmente las problemáticas de un bajo acoplamiento: si bien al abstraer **controllers**, **services** y **repositories** la **Modificabilidad** aumentó, es importante recalcar que el sistema sigue ligado al disco local y a inyecciones asincrónicas en archivos JSON.

Hacia el horizonte del proyecto, es menester la incorporación de un nodo central de persistencia (Base de datos *ACID* o *NoSQL transaccional*), logrando así transicionar los micro-nodos a verdaderos actores *Stateless*. Sólo entonces, será justificable multiplicar dichos procesos detrás de una capa de *proxy inverso* (Nginx), optimizando su **Escalabilidad Horizontal**.

Por otro lado, se prevé inocular nuevas **Métricas Biológicas de Dominio (Business Metrics)** (por ejemplo, el volumen de criptomonedas y moneda local transaccionados acumulativamente en el tiempo, identificando valores absolutos y transacciones netas). Esta nueva incorporación de observabilidad ampliará significativamente los índices de éxito sobre *Business Intelligence*, y se proyectan visualizar de forma unificada sobre *Grafana*.

[NOTA: Una vez implementadas las métricas de dominio (compras/ventas sumadas por divisa e insertadas en Grafana) y reemplazado el archivo JSON por una BD propiamente dicha, no olvidar reescribir esta sección reemplazando el carácter “Parcial” unificando el aprendizaje total que se dio al resolver los limitantes anteriores.]

Referencias

- [1] Node.js Foundation. Node.js. <https://nodejs.org/>, 2026. Accessed: 2026.
- [2] Tim Caswell et al. Node version manager (npm). <https://github.com/creationix/nvm>, 2026. Accessed: 2026.
- [3] Express.js. Hello world example. <https://expressjs.com/en/starter/hello-world.html>, 2026. Accessed: 2026.
- [4] NGINX, Inc. Nginx. <https://nginx.org/>, 2026. Accessed: 2026.
- [5] Redis Ltd. Redis. <https://redis.io/>, 2026. Accessed: 2026.
- [6] npm, Inc. Redis client for node.js. <https://www.npmjs.com/package/redis>, 2026. Accessed: 2026.
- [7] Docker Documentation. Docker engine. <https://docker-k8s-lab.readthedocs.io/en/latest/docker/docker-engine.html>, 2026. Accessed: 2026.
- [8] Docker, Inc. Docker. <https://www.docker.com/>, 2026. Accessed: 2026.
- [9] Docker, Inc. Docker compose. <https://docs.docker.com/compose/>, 2026. Accessed: 2026.
- [10] Etsy. Statsd. <https://github.com/etsy/statsd>, 2026. Accessed: 2026.
- [11] Etsy. Statsd graphite integration. <https://github.com/etsy/statsd/blob/master/docs/graphite.md>, 2026. Accessed: 2026.
- [12] Graphite Project. Graphite. <https://graphiteapp.org/>, 2026. Accessed: 2026.
- [13] Graphite Project. Graphite documentation. <https://graphite.readthedocs.io/en/latest/>, 2026. Accessed: 2026.
- [14] Grafana Labs. Grafana. <https://grafana.com/>, 2026. Accessed: 2026.
- [15] Grafana Labs. Grafana getting started. https://docs.grafana.org/guides/getting_started/, 2026. Accessed: 2026.
- [16] Docker Hub. Graphite statsd image. <https://hub.docker.com/r/graphiteapp/graphite-statsd/>, 2026. Accessed: 2026.
- [17] Graphite Project. Docker graphite statsd. <https://github.com/graphite-project/docker-graphite-statsd>, 2026. Accessed: 2026.
- [18] Dieter Plaetinck. Graphite, grafana, statsd gotchas. <http://dieter.plaetinck.be/post/25-graphite-grafana-statsd-gotchas/>, 2026. Accessed: 2026.
- [19] Artillery.io. Artillery documentation. <https://artillery.io/docs/>, 2026. Accessed: 2026.
- [20] npm, Inc. Artillery package. <https://www.npmjs.com/package/artillery>, 2026. Accessed: 2026.
- [21] npm, Inc. Artillery statsd plugin. <https://www.npmjs.com/package/artillery-plugin-statsd>, 2026. Accessed: 2026.
- [22] Apache Software Foundation. Apache jmeter. <https://jmeter.apache.org/>, 2026. Accessed: 2026.
- [23] Queue-it. Load vs stress testing. <https://queue-it.com/blog/load-vs-stress-testing/>, 2026. Accessed: 2026.
- [24] Artillery.io. Load testing workload models. <https://www.artillery.io/blog/load-testing-workload-models>, 2026. Accessed: 2026.